

✓ Problem Statement

✓ Business Context

Workplace safety in hazardous environments like construction sites and industrial plants is crucial to prevent accidents and injuries. One of the most important safety measures is ensuring workers wear safety helmets, which protect against head injuries from falling objects and machinery. Non-compliance with helmet regulations increases the risk of serious injuries or fatalities, making effective monitoring essential, especially in large-scale operations where manual oversight is prone to errors and inefficiency.

To overcome these challenges, SafeGuard Corp plans to develop an automated image analysis system capable of detecting whether workers are wearing safety helmets. This system will improve safety enforcement, ensuring compliance and reducing the risk of head injuries. By automating helmet monitoring, SafeGuard aims to enhance efficiency, scalability, and accuracy, ultimately fostering a safer work environment while minimizing human error in safety oversight.

✓ Objective

As a data scientist at SafeGuard Corp, you are tasked with developing an image classification model that classifies images into one of two categories:

- **With Helmet:** Workers wearing safety helmets.
- **Without Helmet:** Workers not wearing safety helmets.

✓ Data Description

The dataset consists of **4125 images**, divided into two categories:

- **With Helmet:** 3161 images showing workers wearing helmets.
- **Without Helmet:** 964 images showing workers not wearing helmets.

Dataset Characteristics:

- **Variations in Conditions:** Images include diverse environments such as construction sites, factories, and industrial settings, with variations in lighting, angles, and worker postures to simulate real-world conditions.
- **Worker Activities:** Workers are depicted in different actions such as standing, using tools, or moving, ensuring robust model learning for various scenarios.

Please read the instructions carefully before starting the project.

This is a commented Python Notebook file in which all the instructions and tasks to be performed are mentioned.

- Blanks '*are provided in the notebook that needs to be filled with an appropriate code to get the correct result. With every*' blank, there is a comment that briefly describes what needs to be filled in the blank space.
- Identify the task to be performed correctly, and only then proceed to write the required code.
- Please run the codes in a sequential manner from the beginning to avoid any unnecessary errors.
- Add the results/observations (wherever mentioned) derived from the analysis in the presentation and submit the same. Any mathematical or computational details which are a graded part of the project can be included in the Appendix section of the presentation.

Note: If the free-tier GPU of Google Colab is not accessible (due to unavailability or exhaustion of daily limit or other reasons), the following steps can be taken:

1. Wait for 12-24 hours until the GPU is accessible again or the daily usage limits are reset.
2. Switch to a different Google account and resume working on the project from there.
3. Try using the CPU runtime:
 - To use the CPU runtime, click on *Runtime => Change runtime type => CPU => Save*
 - One can also click on the *Continue without GPU* option to switch to a CPU runtime (kindly refer to the snapshot below)

- The instructions for running the code on the CPU are provided in the respective sections of the notebook.

✓ Installing and Importing the Necessary Libraries

```
!pip install tensorflow[and-cuda]==2.19.0 scikit-learn==1.6.1 opencv-pyt
```

```

_____ 645.0/645.0 MB 2.1 MB/s eta 0
_____ 67.0/67.0 MB 10.5 MB/s eta 0:
_____ 363.3/363.3 MB 5.4 MB/s eta 0
_____ 13.8/13.8 MB 42.6 MB/s eta 0:
_____ 22.5/22.5 MB 61.3 MB/s eta 0:
_____ 24.9/24.9 MB 54.2 MB/s eta 0:
_____ 895.7/895.7 kB 60.2 MB/s eta
_____ 577.2/577.2 MB 3.5 MB/s eta 0
_____ 192.5/192.5 MB 6.6 MB/s eta 0
_____ 56.3/56.3 MB 13.0 MB/s eta 0:
_____ 130.3/130.3 MB 8.8 MB/s eta 0
_____ 217.6/217.6 MB 5.0 MB/s eta 0
_____ 199.0/199.0 MB 8.8 MB/s eta 0
_____ 21.3/21.3 MB 69.0 MB/s eta 0:
_____ 5.5/5.5 MB 71.7 MB/s eta 0:00

```

```

ERROR: pip's dependency resolver does not currently take into account all
ydf-tf 2.20.0 requires tensorflow==2.20.0, but you have tensorflow 2.19.0
tf-keras 2.20.0 requires tensorflow<2.21,>=2.20, but you have tensorflow
tensorflow-text 2.20.1 requires tensorflow<2.21,>=2.20.0, but you have te
torch 2.10.0+cu128 requires nvidia-cublas-cu12==12.8.4.1; platform_system
torch 2.10.0+cu128 requires nvidia-cuda-cupti-cu12==12.8.90; platform_sys
torch 2.10.0+cu128 requires nvidia-cuda-nvrtc-cu12==12.8.93; platform_sys
torch 2.10.0+cu128 requires nvidia-cuda-runtime-cu12==12.8.90; platform_s
torch 2.10.0+cu128 requires nvidia-cudnn-cu12==9.10.2.21; platform_system
torch 2.10.0+cu128 requires nvidia-cufft-cu12==11.3.3.83; platform_system
torch 2.10.0+cu128 requires nvidia-curand-cu12==10.3.9.90; platform_syste
torch 2.10.0+cu128 requires nvidia-cusolver-cu12==11.7.3.90; platform_sys
torch 2.10.0+cu128 requires nvidia-cuspars-cu12==12.5.8.93; platform_sys
torch 2.10.0+cu128 requires nvidia-nccl-cu12==2.27.5; platform_system ==
torch 2.10.0+cu128 requires nvidia-nvjitlink-cu12==12.8.93; platform_syst

```

Note:

- After running the above cell, kindly restart the notebook kernel (for Jupyter Notebook) or runtime (for Google Colab) and run all cells sequentially from the next cell.
- On executing the above line of code, you might see a warning regarding package dependencies. This error message can be ignored as the above code ensures that all necessary libraries and their dependencies are maintained to successfully execute the code in this notebook.

```
import os
import random
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.image as mpimg
import matplotlib.pyplot as plt
import math
import cv2

# Tensorflow modules
import keras
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, Flatten, Conv2D, MaxPool
from tensorflow.keras.optimizers import Adam, SGD
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix
from tensorflow.keras.models import Model
from keras.applications.vgg16 import VGG16

# Display images using OpenCV
from google.colab.patches import cv2_imshow

#Imports functions for evaluating the performance of machine learning mo
from sklearn.metrics import confusion_matrix, f1_score, accuracy_score, r
from sklearn.metrics import mean_squared_error as mse

# Ignore warnings
import warnings
warnings.filterwarnings('ignore')
```

```
print("Num GPUs Available:", len(tf.config.list_physical_devices('GPU')))
```

```
Num GPUs Available: 1
```

```
# 1. Set Python random seed
random.seed(812)

# 2. Set NumPy random seed
np.random.seed(812)

# 3. Set TensorFlow seed (covers Keras + backend)
tf.keras.utils.set_random_seed(812)

# 4. Enable deterministic GPU ops (if using GPU)
tf.config.experimental.enable_op_determinism()
```

✓ Data Overview

✓ Loading the data

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
# Uncomment and run the following code in case Google Colab is being use
# from google.colab import drive
# drive.mount('/content/drive')
```

```
images = np.load('images.npy') #Complete the code to load the images

labels = pd.read_csv('labels.csv') #Complete the code to load the labels
```

```
print(images.shape) #Complete the code to print the shape of the images
print(labels.shape) #Complete the code to print the shape of the labels
```

```
(4125, 200, 200, 3)
(4125, 1)
```

✓ Exploratory Data Analysis

✓

Plot random images from each of the classes and print their corresponding labels.

```
# Get indices for 'With Helmet' (label 1) and 'Without Helmet' (label 0)
helmet_indices = labels[labels.iloc[:, 0] == 1].index.tolist()
no_helmet_indices = labels[labels.iloc[:, 0] == 0].index.tolist()

# Select 5 random indices from each class
num_samples_to_plot = 5
random_helmet_idx = random.sample(helmet_indices, num_samples_to_plot)
random_no_helmet_idx = random.sample(no_helmet_indices, num_samples_to_plot)

fig, axes = plt.subplots(num_samples_to_plot, 2, figsize=(10, 4 * num_sa

for i in range(num_samples_to_plot):
    # Plot 'With Helmet' images
    axes[i][0].imshow(images[random_helmet_idx[i]])
    axes[i][0].set_title(f"With Helmet (Index: {random_helmet_idx[i]})")
    axes[i][0].axis("off")

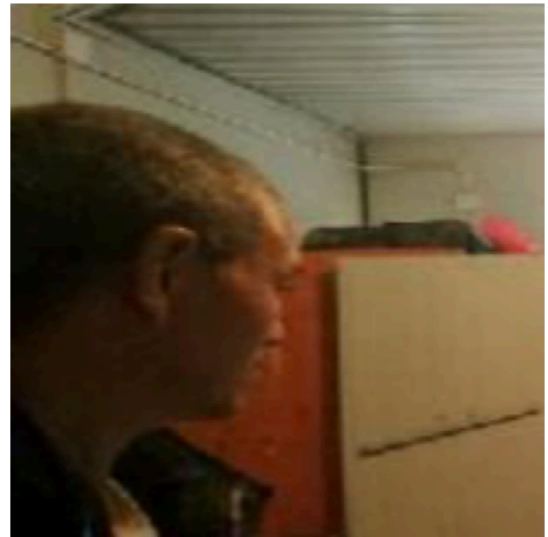
    # Plot 'Without Helmet' images
    axes[i][1].imshow(images[random_no_helmet_idx[i]])
    axes[i][1].set_title(f"Without Helmet (Index: {random_no_helmet_idx[i]})")
    axes[i][1].axis("off")

plt.tight_layout()
plt.show()
```


With Helmet (Index: 1092)



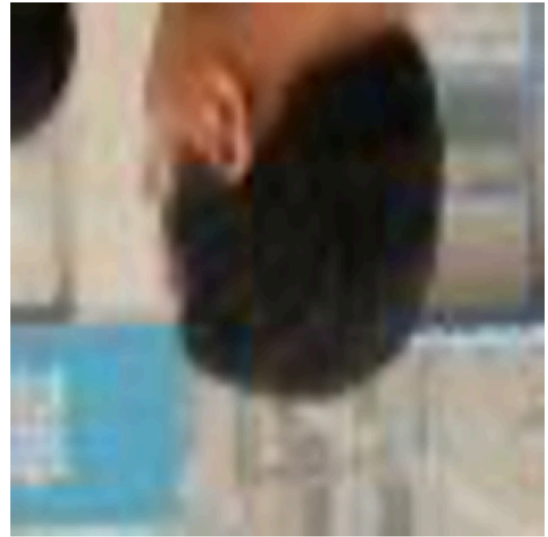
Without Helmet (Index: 360)



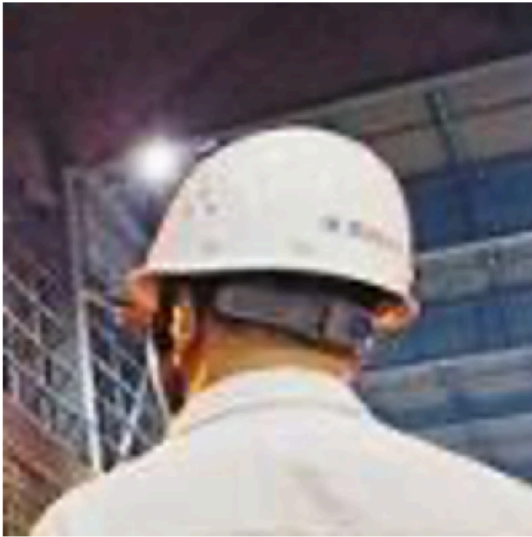
With Helmet (Index: 2590)



Without Helmet (Index: 660)



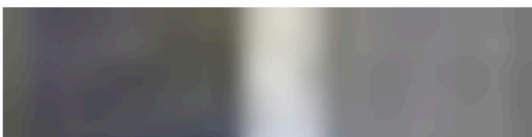
With Helmet (Index: 3477)



Without Helmet (Index: 835)

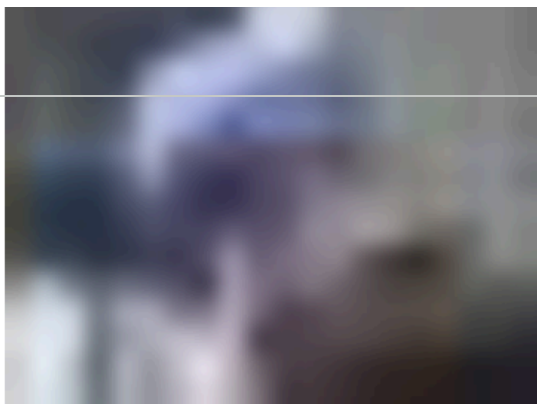


With Helmet (Index: 3892)

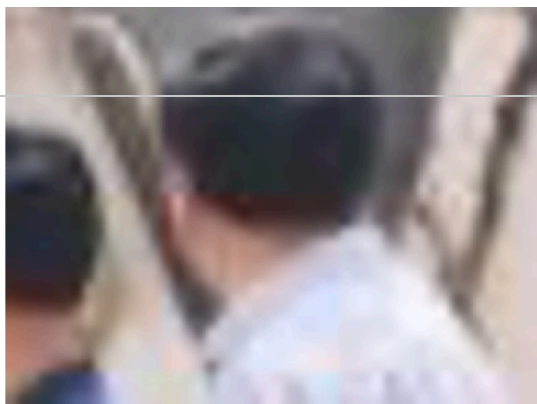


Without Helmet (Index: 782)





With Helmet (Index: 2173)



Without Helmet (Index: 289)



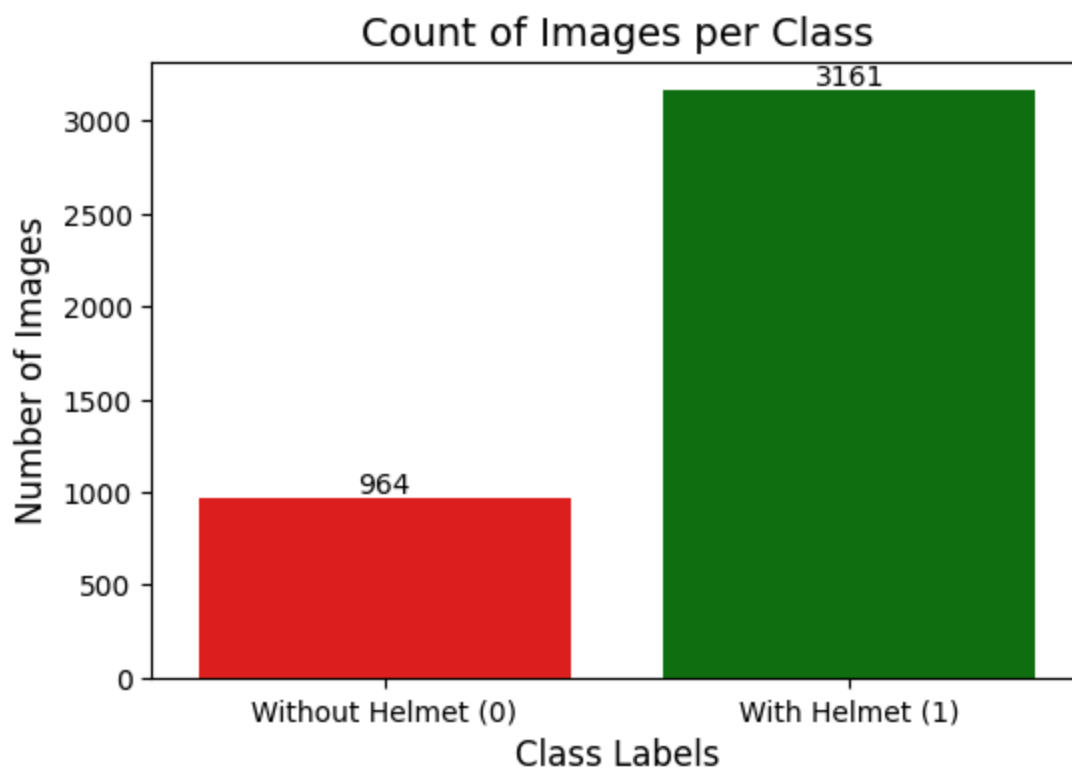
✓ Checking for class imbalance

```
# Create a count plot
plt.figure(figsize=(6, 4))
ax = sns.countplot(x=labels.iloc[:, 0], palette=['red', 'green'])

# Add exact counts on top of bars
for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}', (p.get_x() + p.get_width() / 2
                                         ha='center', va='bottom', fontsize=10, ))

# Add labels
plt.xlabel("Class Labels", fontsize=12)
plt.ylabel("Number of Images", fontsize=12)
plt.title("Count of Images per Class", fontsize=14)
plt.xticks(ticks=[0, 1], labels=["Without Helmet (0)", "With Helmet (1)"])

# Show plot
plt.show()
```



✓ Data Preprocessing

✓ Splitting the dataset

```
X_train, X_temp, y_train, y_temp = train_test_split(np.array(images), labels,
X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.2)
```

```
print(X_train.shape, y_train.shape) #Complete the code to print the shape of X_train
print(X_val.shape, y_val.shape) #Complete the code to print the shape of X_val
print(X_test.shape, y_test.shape) #Complete the code to print the shape of X_test
```

```
(2887, 200, 200, 3) (2887, 1)
(619, 200, 200, 3) (619, 1)
(619, 200, 200, 3) (619, 1)
```

✓ Data Normalization

Since the **image pixel values range from 0-255**, our method of normalization here will be **scaling** - we shall **divide all the pixel values by 255 to standardize the images to have values between 0-1**.

```
X_train_normalized = X_train.astype('float32')/255 #Complete the code to normalize X_train
X_val_normalized = X_val.astype('float32')/255 #Complete the code to normalize X_val
X_test_normalized = X_test.astype('float32')/255 #Complete the code to normalize X_test
```

✓ Model Building

✓ Utility Functions

```
# defining a function to compute different metrics to check performance
def model_performance_classification(model, predictors, target):
    """
    Function to compute different metrics to check classification model performance

    model: classifier
    predictors: independent variables
    target: dependent variable
    """

    # checking which probabilities are greater than threshold
    pred = model.predict(predictors).reshape(-1)>0.5

    target = target.to_numpy().reshape(-1)

    acc = accuracy_score(target, pred) # to compute Accuracy
```

```

recall = recall_score(target, pred, average='macro') # to compute R
precision = precision_score(target, pred, average='macro') # to com
f1 = f1_score(target, pred, average='macro') # to compute F1-score

# creating a dataframe of metrics
df_perf = pd.DataFrame({"Accuracy": acc, "Recall": recall, "Precisio

return df_perf

```

```

def plot_confusion_matrix(model, predictors, target, ml=False):
    """
    Function to plot the confusion matrix

    model: classifier
    predictors: independent variables
    target: dependent variable
    ml: To specify if the model used is an sklearn ML model or not (True
    """

    # checking which probabilities are greater than threshold
    pred = model.predict(predictors).reshape(-1)>0.5

    target = target.to_numpy().reshape(-1)

    # Plotting the Confusion Matrix using confusion matrix() function wh
    confusion_matrix = tf.math.confusion_matrix(target, pred)
    f, ax = plt.subplots(figsize=(10, 8))
    sns.heatmap(
        confusion_matrix,
        annot=True,
        linewidths=.4,
        fmt="d",
        square=True,
        ax=ax
    )
    plt.show()

```

✓ Model 1: Convolutional Neural Network (CNN) from Scratch

Let's build a **CNN Model**.

The model has 2 main parts:

1. **The Feature Extraction layers** which are comprised of convolutional and pooling layers.
2. **The Fully Connected classification layers for prediction.**

```
# Initializing Model
model_1 = Sequential()

# Convolutional layers
model_1.add(Conv2D(16, (3, 3), activation='relu', padding='same', input_shape=(28, 28, 1)))
model_1.add(MaxPooling2D((4, 4), padding='same'))
model_1.add(Conv2D(64, (3, 3), activation='relu', padding='same')) #Complete the code to define the number of filters
model_1.add(MaxPooling2D((2,2), padding='same')) #Complete the code to define the number of filters
model_1.add(Conv2D(128, (3,3), activation='relu', padding='same')) #Complete the code to define the number of filters

# Flatten and Dense layers
model_1.add(Flatten())
model_1.add(Dense(4, activation='relu'))
model_1.add(Dense(1, activation='sigmoid')) #Complete the code to define the number of units
```

```
# Compile with Adam Optimizer
opt = Adam(learning_rate=0.001) #Complete the code to define the loss function and metrics
```

```
model_1.compile(optimizer=opt, loss='binary_crossentropy', metrics=['accuracy'])
```

```
# Summary
model_1.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param
conv2d_3 (Conv2D)	(None, 200, 200, 16)	44
max_pooling2d_2 (MaxPooling2D)	(None, 50, 50, 16)	
conv2d_4 (Conv2D)	(None, 50, 50, 64)	9,28
max_pooling2d_3 (MaxPooling2D)	(None, 25, 25, 64)	
conv2d_5 (Conv2D)	(None, 25, 25, 128)	73,85
flatten_1 (Flatten)	(None, 80000)	
dense_2 (Dense)	(None, 4)	320,00
dense_3 (Dense)	(None, 1)	

Total params: 403,593 (1.54 MB)
Trainable params: 403,593 (1.54 MB)
Non-trainable params: 0 (0.00 B)

```
history_1 = model_1.fit(
    X_train_normalized, y_train,
    epochs=20, #Complete the code to define the number of epochs
    validation_data=(X_val_normalized, y_val),
```

```

shuffle=True,
batch_size=32, #Complete the code to define the batch size
verbose=2
)

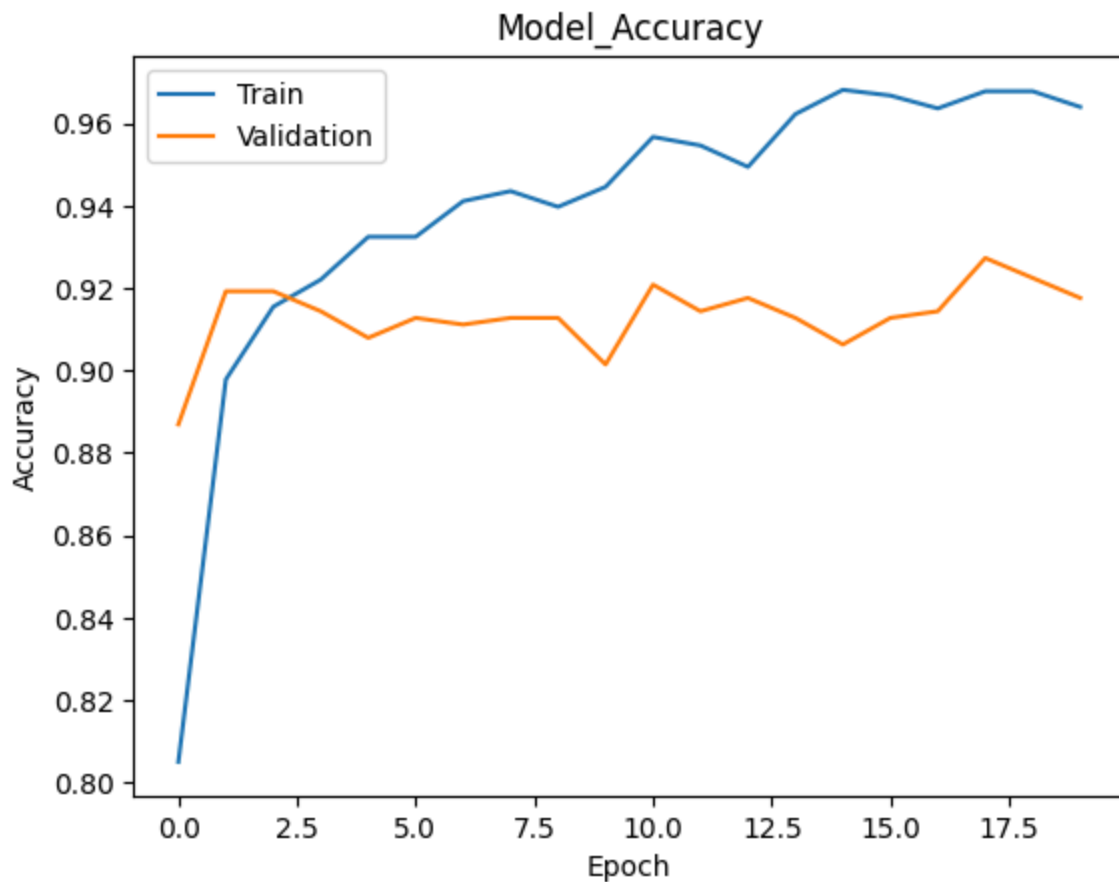
Epoch 1/20
91/91 - 7s - 73ms/step - Recall: 0.9792 - accuracy: 0.8050 - loss: 0.4064
Epoch 2/20
91/91 - 3s - 34ms/step - Recall: 0.9525 - accuracy: 0.8978 - loss: 0.2907
Epoch 3/20
91/91 - 2s - 27ms/step - Recall: 0.9571 - accuracy: 0.9155 - loss: 0.2498
Epoch 4/20
91/91 - 2s - 27ms/step - Recall: 0.9552 - accuracy: 0.9221 - loss: 0.2385
Epoch 5/20
91/91 - 2s - 27ms/step - Recall: 0.9625 - accuracy: 0.9325 - loss: 0.2229
Epoch 6/20
91/91 - 3s - 31ms/step - Recall: 0.9575 - accuracy: 0.9325 - loss: 0.2108
Epoch 7/20
91/91 - 3s - 33ms/step - Recall: 0.9602 - accuracy: 0.9411 - loss: 0.2039
Epoch 8/20
91/91 - 3s - 28ms/step - Recall: 0.9611 - accuracy: 0.9435 - loss: 0.1920
Epoch 9/20
91/91 - 2s - 27ms/step - Recall: 0.9503 - accuracy: 0.9397 - loss: 0.1950
Epoch 10/20
91/91 - 3s - 29ms/step - Recall: 0.9593 - accuracy: 0.9446 - loss: 0.1816
Epoch 11/20
91/91 - 3s - 28ms/step - Recall: 0.9688 - accuracy: 0.9567 - loss: 0.1591
Epoch 12/20
91/91 - 3s - 32ms/step - Recall: 0.9670 - accuracy: 0.9546 - loss: 0.1582
Epoch 13/20
91/91 - 3s - 28ms/step - Recall: 0.9634 - accuracy: 0.9494 - loss: 0.1603
Epoch 14/20
91/91 - 2s - 27ms/step - Recall: 0.9724 - accuracy: 0.9622 - loss: 0.1373
Epoch 15/20
91/91 - 2s - 26ms/step - Recall: 0.9729 - accuracy: 0.9681 - loss: 0.1277
Epoch 16/20
91/91 - 2s - 27ms/step - Recall: 0.9711 - accuracy: 0.9667 - loss: 0.1294
Epoch 17/20
91/91 - 3s - 31ms/step - Recall: 0.9711 - accuracy: 0.9636 - loss: 0.1254
Epoch 18/20
91/91 - 3s - 30ms/step - Recall: 0.9711 - accuracy: 0.9678 - loss: 0.1192
Epoch 19/20
91/91 - 3s - 28ms/step - Recall: 0.9720 - accuracy: 0.9678 - loss: 0.1186
Epoch 20/20
91/91 - 3s - 29ms/step - Recall: 0.9679 - accuracy: 0.9640 - loss: 0.1256

```

```

plt.plot(history_1.history['accuracy']) #Complete the code to plot the t
plt.plot(history_1.history['val_accuracy']) #Complete the code to plot t
plt.title('Model_Accuracy') #Complete the code to define the title for t
plt.ylabel('Accuracy') #Complete the code to define the label for the y-
plt.xlabel('Epoch') #Complete the code to define the label for the x-axi
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()

```

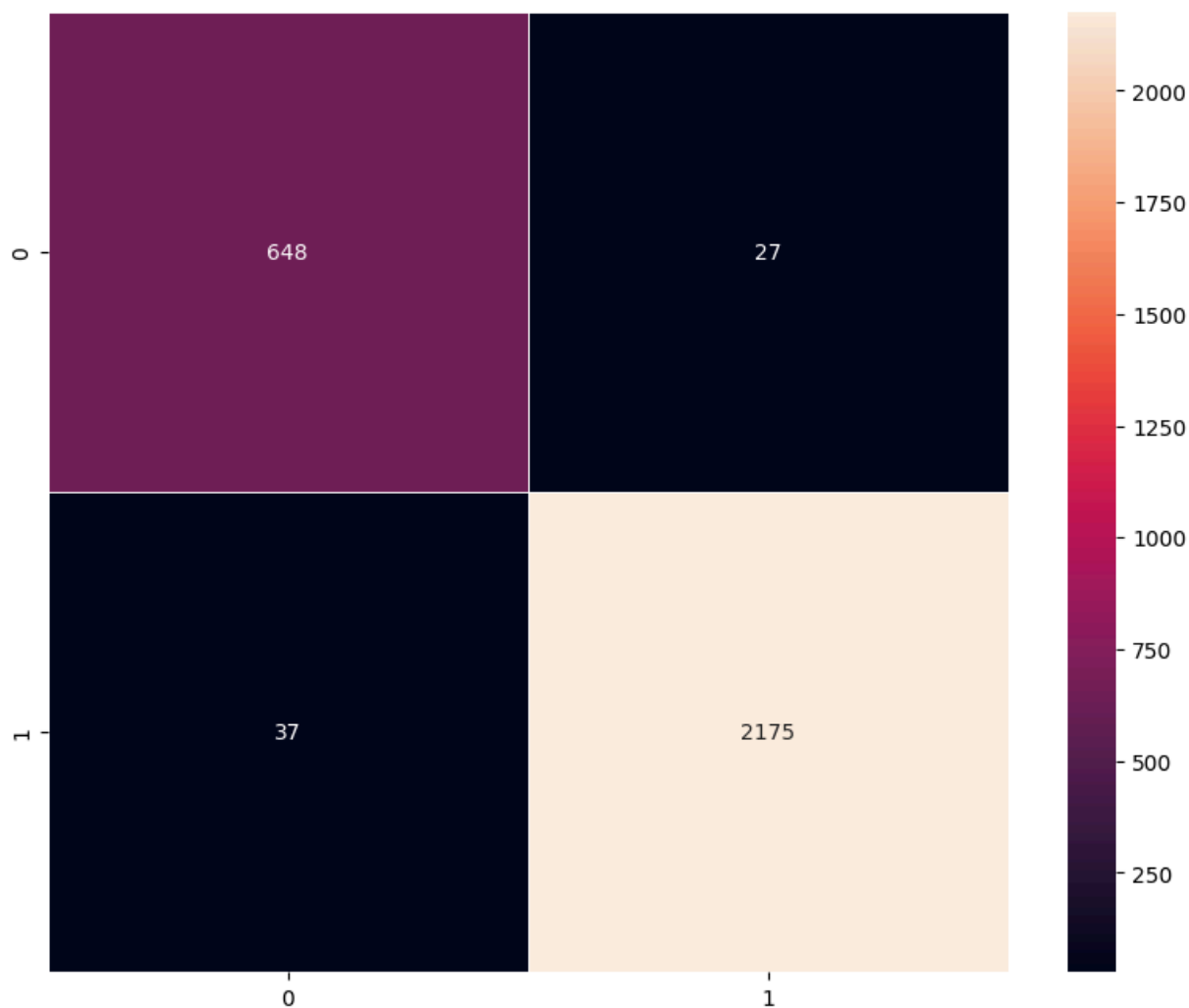


```
model_1_train_perf = model_performance_classification(model_1, X_train_n  
  
print("Train performance metrics")  
print(model_1_train_perf)
```

91/91 ————— **3s** 18ms/step
Train performance metrics
Accuracy Recall Precision F1 Score
0 0.977832 0.971637 0.966862 0.969221

```
plot_confusion_matrix(model_1,X_train_normalized,y_train)
```

91/91 ————— 2s 16ms/step



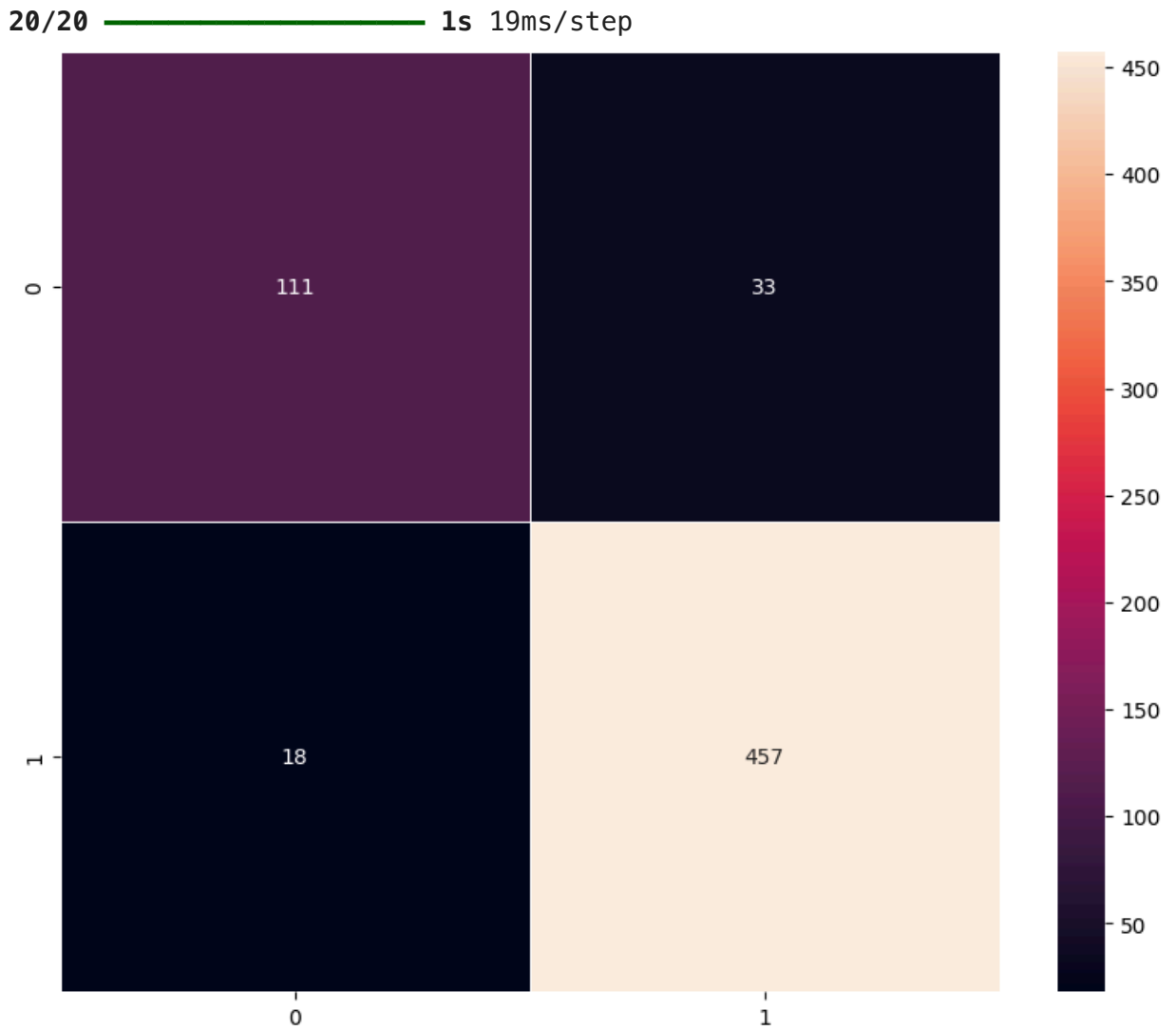
```
model_1_valid_perf = model_performance_classification(model_1, X_val_nor
print("Validation performance metrics")
print(model_1_valid_perf)
```

20/20 ————— 1s 19ms/step

Validation performance metrics

	Accuracy	Recall	Precision	F1 Score
0	0.917609	0.866469	0.896559	0.880169

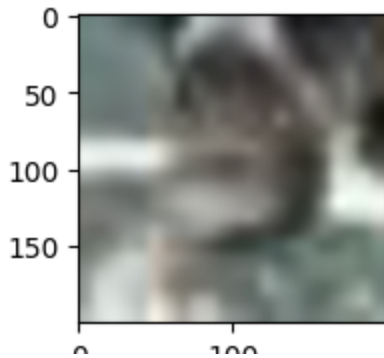
```
plot_confusion_matrix(model_1,X_val_normalized,y_val)
```

✓ Visualizing the predictions

```
# For index 12
plt.figure(figsize=(2,2))
plt.imshow(X_val[12])
plt.show()
prediction = model_1.predict(X_val_normalized[12].reshape(1, 200, 200, 3))
predicted_label = prediction[0][0]>0.5 # Extract the predicted class label
print('Predicted Label:', 1 if predicted_label else 0)
# Fix indexing issue in y_val
true_label = y_val.iloc[12]
print('True Label:', true_label)
```

```
# For index 33
plt.figure(figsize=(2,2))
plt.imshow(X_val[33])
plt.show()
prediction = model_1.predict(X_val_normalized[12].reshape(1, 200, 200, 3)
predicted_label = prediction[0][0]>0.5 # Extract the predicted class la
print('Predicted Label:', 1 if predicted_label else 0)
# Fix indexing issue in y_val
true_label = y_val.iloc[33]
print('True Label:', true_label)
```

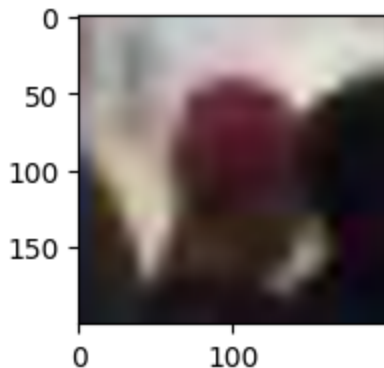


1/1  0s 89ms/step

Predicted Label: 1

True Label: label 0

Name: 840, dtype: int64



1/1  0s 47ms/step

Predicted Label: 1

True Label: label 1

Name: 1133, dtype: int64

✓ Model 2: Transfer Learning with VGG-16 (Base)

- We will be loading a pre-built architecture - **VGG16**, which was trained on the ImageNet dataset and is the runner-up in the ImageNet competition in 2014.

- For training VGG16, we will directly use the convolutional and pooling layers and freeze their weights i.e. no training will be done on them. For classification, we will add a Flatten and a single dense layer.

```
vgg_model = VGG16(weights='imagenet', include_top=False, input_shape=(200,
vgg_model.summary())
```

Downloading data from <https://storage.googleapis.com/tensorflow/keras-apps/58889256/58889256>  2s 0us/step

Model: "vgg16"

Layer (type)	Output Shape	Param
input_layer_2 (InputLayer)	(None, 200, 200, 3)	
block1_conv1 (Conv2D)	(None, 200, 200, 64)	1,79
block1_conv2 (Conv2D)	(None, 200, 200, 64)	36,92
block1_pool (MaxPooling2D)	(None, 100, 100, 64)	
block2_conv1 (Conv2D)	(None, 100, 100, 128)	73,85
block2_conv2 (Conv2D)	(None, 100, 100, 128)	147,58
block2_pool (MaxPooling2D)	(None, 50, 50, 128)	
block3_conv1 (Conv2D)	(None, 50, 50, 256)	295,16
block3_conv2 (Conv2D)	(None, 50, 50, 256)	590,08
block3_conv3 (Conv2D)	(None, 50, 50, 256)	590,08
block3_pool (MaxPooling2D)	(None, 25, 25, 256)	
block4_conv1 (Conv2D)	(None, 25, 25, 512)	1,180,16
block4_conv2 (Conv2D)	(None, 25, 25, 512)	2,359,80
block4_conv3 (Conv2D)	(None, 25, 25, 512)	2,359,80
block4_pool (MaxPooling2D)	(None, 12, 12, 512)	
block5_conv1 (Conv2D)	(None, 12, 12, 512)	2,359,80
block5_conv2 (Conv2D)	(None, 12, 12, 512)	2,359,80
block5_conv3 (Conv2D)	(None, 12, 12, 512)	2,359,80
block5_pool (MaxPooling2D)	(None, 6, 6, 512)	

Total params: 14,714,688 (56.13 MB)

Trainable params: 14,714,688 (56.13 MB)

```
# Making all the layers of the VGG model non-trainable. i.e. freezing th
for layer in vgg_model.layers:
    layer.trainable = False
```

```
model_2 = Sequential()

# Adding the convolutional part of the VGG16 model from above
model_2.add(vgg_model)

# Flattening the output of the VGG16 model because it is from a convolut
model_2.add(Flatten())

# Adding a dense output layer
model_2.add(Dense(1, activation='sigmoid')) #Complete the code to define
```

```
opt=Adam(learning_rate=0.0001) #Complete the code to define the learning
# Compile model
model_2.compile(optimizer=opt, loss=keras.losses.BinaryCrossentropy(), m
```

```
# Generating the summary of the model
model_2.summary()
```

Model: "sequential_3"

Layer (type)	Output Shape	Param
vgg16 (Functional)	(None , 6, 6, 512)	14,714,68
flatten_3 (Flatten)	(None , 18432)	
dense_5 (Dense)	(None , 1)	18,43

Total params: 14,733,121 (56.20 MB)
Trainable params: 18,433 (72.00 KB)
Non-trainable params: 14,714,688 (56.13 MB)

```
train_datagen = ImageDataGenerator()
```

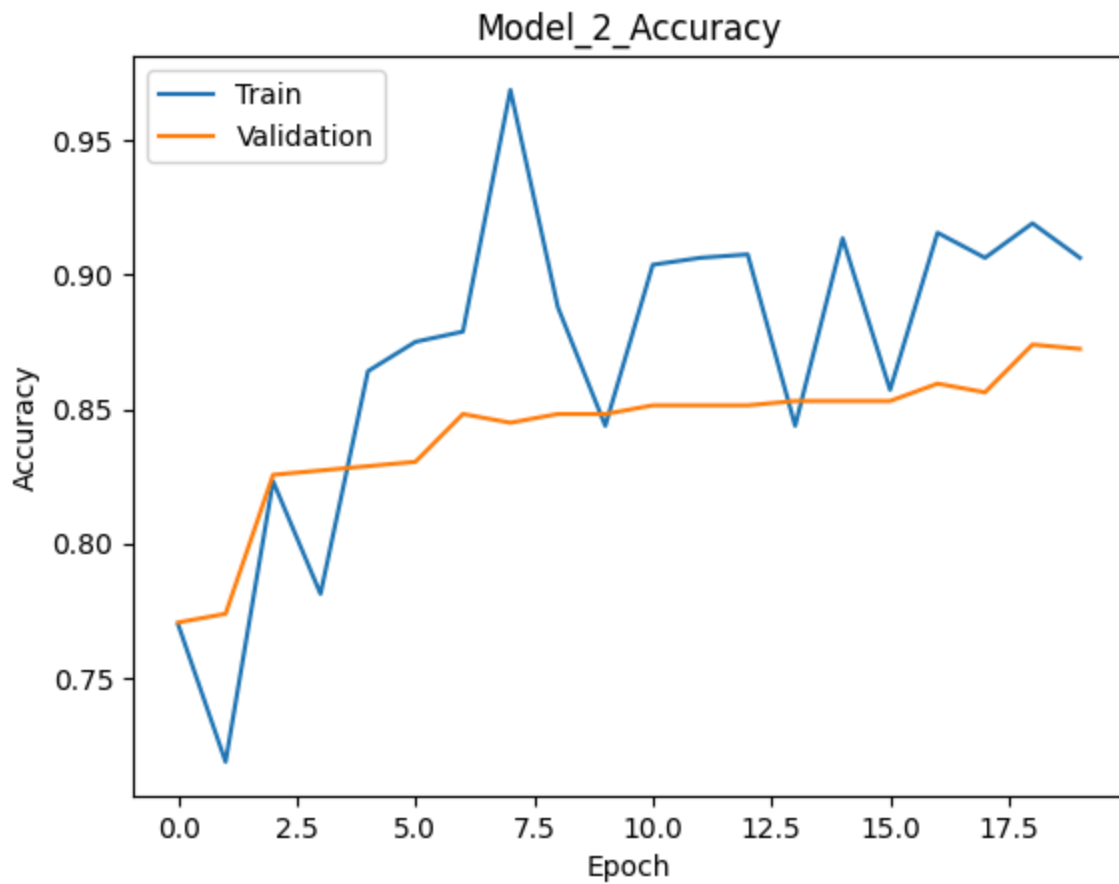
```
# Epochs
epochs = 20 #Complete the code to define the epochs
# Batch size
batch_size = 32 #Complete the code to define the batch size

history_2 = model_2.fit(train_datagen.flow(X_train_normalized,y_train,
                                          batch_size=batch_size,
                                          seed=42,
                                          shuffle=False),
                      epochs=epochs,
                      steps_per_epoch=X_train_normalized.shape[0] // batch
```

```
validation_data=(X_val_normalized,y_val),
verbose=1)
```

```
Epoch 1/20
90/90  16s 160ms/step - accuracy: 0.7699 - loss: 0.47
Epoch 2/20
90/90  2s 26ms/step - accuracy: 0.7188 - loss: 0.5027
Epoch 3/20
90/90  14s 151ms/step - accuracy: 0.8231 - loss: 0.38
Epoch 4/20
90/90  3s 28ms/step - accuracy: 0.7812 - loss: 0.4048
Epoch 5/20
90/90  15s 169ms/step - accuracy: 0.8641 - loss: 0.33
Epoch 6/20
90/90  3s 32ms/step - accuracy: 0.8750 - loss: 0.3013
Epoch 7/20
90/90  15s 164ms/step - accuracy: 0.8788 - loss: 0.31
Epoch 8/20
90/90  3s 30ms/step - accuracy: 0.9688 - loss: 0.2301
Epoch 9/20
90/90  16s 172ms/step - accuracy: 0.8879 - loss: 0.28
Epoch 10/20
90/90  3s 30ms/step - accuracy: 0.8438 - loss: 0.3416
Epoch 11/20
90/90  15s 166ms/step - accuracy: 0.9037 - loss: 0.27
Epoch 12/20
90/90  3s 30ms/step - accuracy: 0.9062 - loss: 0.2144
Epoch 13/20
90/90  17s 194ms/step - accuracy: 0.9075 - loss: 0.25
Epoch 14/20
90/90  3s 30ms/step - accuracy: 0.8438 - loss: 0.3334
Epoch 15/20
90/90  15s 169ms/step - accuracy: 0.9135 - loss: 0.24
Epoch 16/20
90/90  3s 30ms/step - accuracy: 0.8571 - loss: 0.2697
Epoch 17/20
90/90  16s 173ms/step - accuracy: 0.9156 - loss: 0.23
Epoch 18/20
90/90  3s 32ms/step - accuracy: 0.9062 - loss: 0.1963
Epoch 19/20
90/90  16s 173ms/step - accuracy: 0.9191 - loss: 0.22
Epoch 20/20
90/90  3s 32ms/step - accuracy: 0.9062 - loss: 0.2657
```

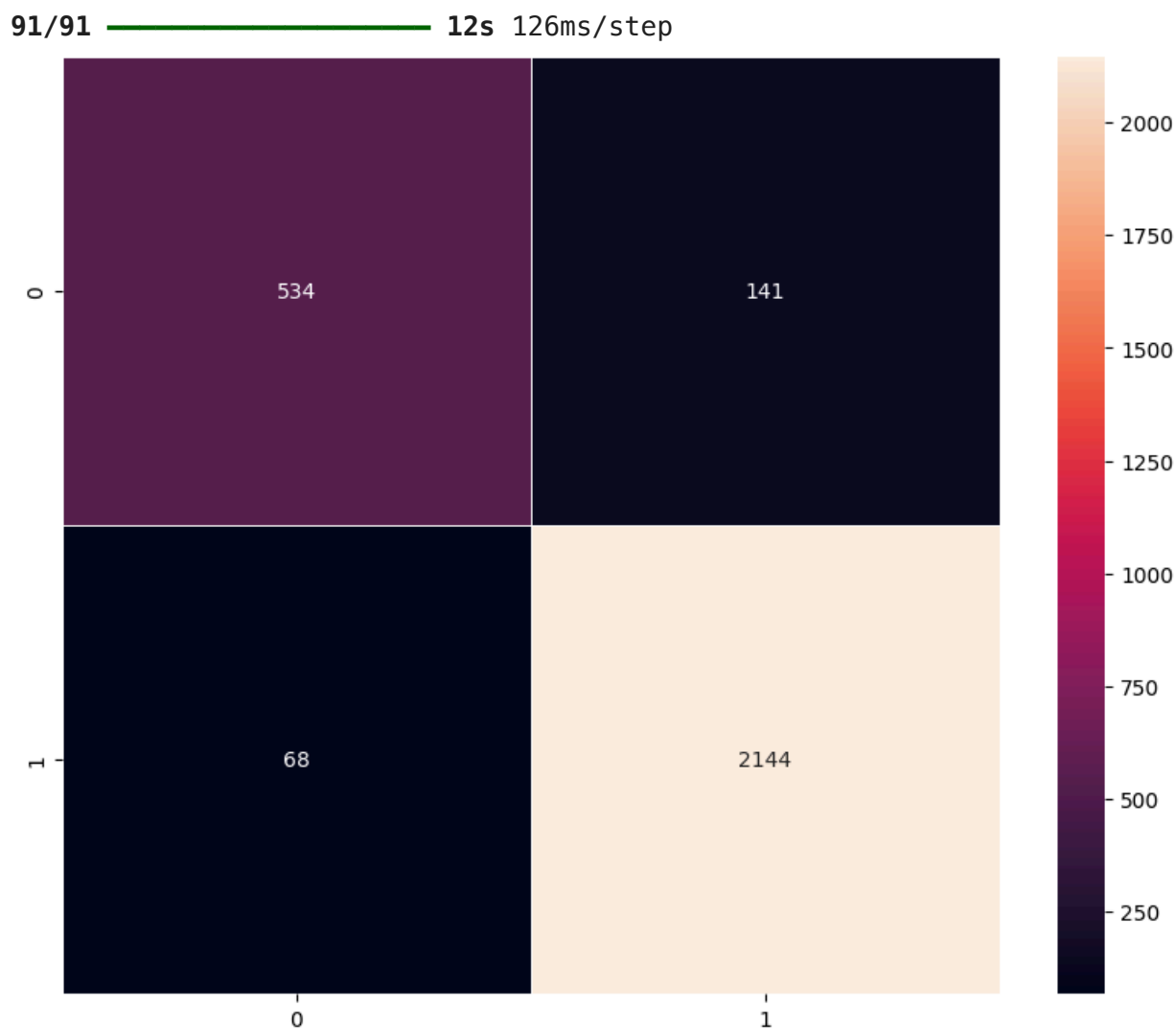
```
plt.plot(history_2.history['accuracy']) #Complete the code to plot the t
plt.plot(history_2.history['val_accuracy']) #Complete the code to plot t
plt.title('Model_2_Accuracy') #Complete the code to define the title for
plt.ylabel('Accuracy') #Complete the code to define the label for the y-
plt.xlabel('Epoch') #Complete the code to define the label for the x-axi
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```



```
model_2_train_perf = model_performance_classification(model_2,X_train_no  
print("Train performance metrics")  
print(model_2_train_perf)
```

91/91 ————— **14s** 137ms/step
Train performance metrics
Accuracy Recall Precision F1 Score
0 0.927607 0.880185 0.912668 0.89493

```
plot_confusion_matrix(model_2,X_train_normalized,y_train)
```



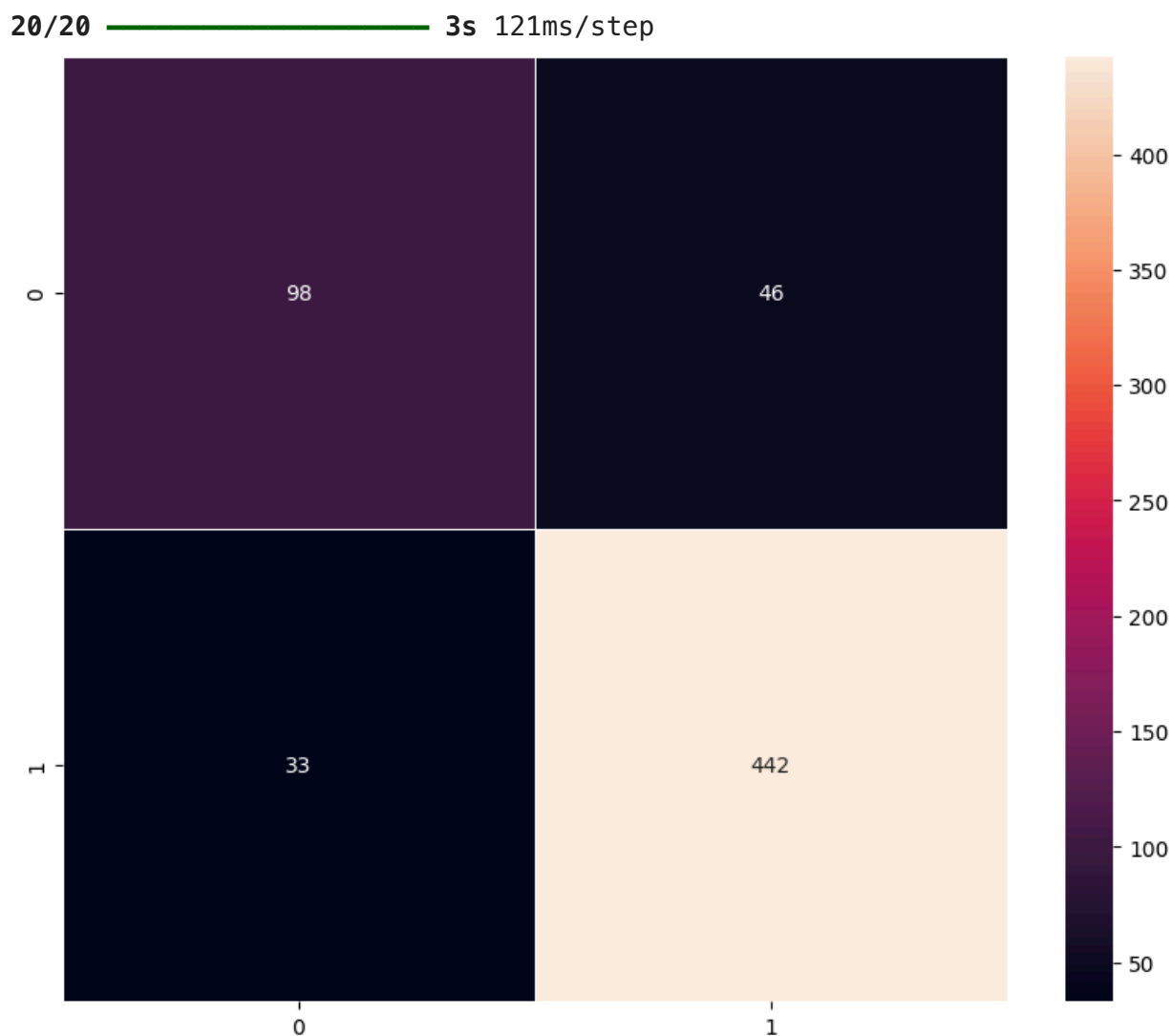
```
model_2_valid_perf = model_performance_classification(model_2, X_val_norm)
print("Validation performance metrics")
print(model_2_valid_perf)
```

20/20 ————— 3s 122ms/step

Validation performance metrics

	Accuracy	Recall	Precision	F1 Score
0	0.872375	0.805541	0.826915	0.815346

```
plot_confusion_matrix(model_2,X_val_normalized,y_val)
```

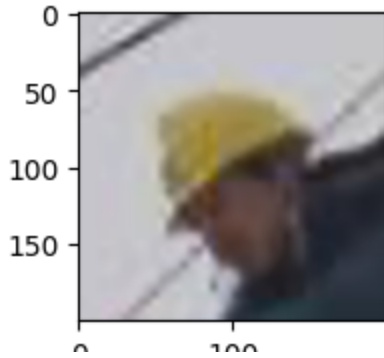


✓ Visualizing the predictions

```
plt.figure(figsize=(2,2))
plt.imshow(X_val[0]) #Complete the code to define the index
plt.show()
prediction = model_2.predict(X_val_normalized[0].reshape(1,200,200,3)) #
predicted_label = prediction[0][0]>0.5 # Extract the predicted class la
print('Predicted Label:', 1 if predicted_label else 0)
# Fix indexing issue in y_val
true_label = y_val.iloc[0] #Complete the code to define the index
print('True Label:', true_label)
```



```
plt.figure(figsize=(2,2))
plt.imshow(X_val[1]) #Complete the code to define the index
plt.show()
prediction = model_2.predict(X_val_normalized[1].reshape(1,200,200,3)) #
predicted_label = prediction[0][0]>0.5 # Extract the predicted class la
print('Predicted Label:', 1 if predicted_label else 0)
# Fix indexing issue in y_val
true_label = y_val.iloc[1] #Complete the code to define the index
print('True Label:', true_label)
```

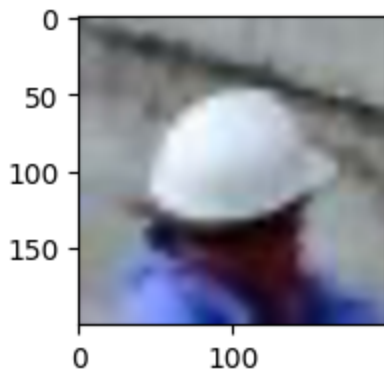


1/1 ————— 0s 102ms/step

Predicted Label: 1

True Label: label 1

Name: 2735, dtype: int64



1/1 ————— 0s 61ms/step

Predicted Label: 1

True Label: label 1

Name: 4006, dtype: int64

✓ Model 3: Transfer Learning with VGG-16 (Base + FFNN)

- We will directly use the convolutional and pooling layers and freeze their weights i.e. no training will be done on them. For classification, we will add a Flatten layer and a Feed Forward Neural Network.

```
model_3 = Sequential()
```

```
# Adding the convolutional part of the VGG16 model from above
model_3.add(vgg_model)

# Flattening the output of the VGG16 model because it is from a convolut
model_3.add(Flatten())

#Adding the Feed Forward neural network
model_3.add(Dense(256,activation='relu')) #Complete the code to define t
model_3.add(Dropout(rate=0.5)) #Complete the code to define the dropout
model_3.add(Dense(128,activation='relu')) #Complete the code to define t

# Adding a dense output layer
model_3.add(Dense(1, activation='sigmoid')) #Complete the code to define
```

```
opt = Adam(learning_rate=0.001) #Complete the code to define the learnin
```

```
# Compile model
model_3.compile(optimizer=opt,loss=keras.losses.BinaryCrossentropy(), me
```

```
# Generating the summary of the model
model_3.summary()
```

Model: "sequential_4"

Layer (type)	Output Shape	Param
vgg16 (Functional)	(None , 6, 6, 512)	14,714,68
flatten_4 (Flatten)	(None , 18432)	
dense_6 (Dense)	(None , 256)	4,718,84
dropout (Dropout)	(None , 256)	
dense_7 (Dense)	(None , 128)	32,89
dense_8 (Dense)	(None , 1)	12

Total params: 19,466,561 (74.26 MB)
Trainable params: 4,751,873 (18.13 MB)
Non-trainable params: 14,714,688 (56.13 MB)

```
history_3 = model_3.fit(train_datagen.flow(X_train_normalized,y_train,
                                           batch_size=32, #Complete the code
                                           seed=42,
                                           shuffle=False),
                      epochs=25, #Complete the code to define the number o
                      steps_per_epoch=X_train_normalized.shape[0] // batch
                      validation_data=(X_val_normalized,y_val),
                      verbose=1)
```

```

Epoch 1/25
90/90  18s 165ms/step - accuracy: 0.7503 - loss: 0.57
Epoch 2/25
90/90  2s 26ms/step - accuracy: 0.8750 - loss: 0.3549
Epoch 3/25
90/90  14s 151ms/step - accuracy: 0.8623 - loss: 0.30
Epoch 4/25
90/90  3s 27ms/step - accuracy: 0.8750 - loss: 0.2398
Epoch 5/25
90/90  14s 154ms/step - accuracy: 0.8729 - loss: 0.29
Epoch 6/25
90/90  3s 28ms/step - accuracy: 0.9062 - loss: 0.3397
Epoch 7/25
90/90  14s 159ms/step - accuracy: 0.8837 - loss: 0.26
Epoch 8/25
90/90  3s 29ms/step - accuracy: 0.8571 - loss: 0.2250
Epoch 9/25
90/90  17s 189ms/step - accuracy: 0.8907 - loss: 0.24
Epoch 10/25
90/90  3s 31ms/step - accuracy: 0.8750 - loss: 0.2628
Epoch 11/25
90/90  15s 168ms/step - accuracy: 0.8893 - loss: 0.23
Epoch 12/25
90/90  3s 30ms/step - accuracy: 0.9062 - loss: 0.3587
Epoch 13/25
90/90  15s 168ms/step - accuracy: 0.8893 - loss: 0.24
Epoch 14/25
90/90  3s 30ms/step - accuracy: 0.9375 - loss: 0.1775
Epoch 15/25
90/90  15s 172ms/step - accuracy: 0.8949 - loss: 0.22
Epoch 16/25
90/90  3s 31ms/step - accuracy: 0.8438 - loss: 0.3826
Epoch 17/25
90/90  16s 176ms/step - accuracy: 0.9145 - loss: 0.19
Epoch 18/25
90/90  3s 31ms/step - accuracy: 0.8750 - loss: 0.2712
Epoch 19/25
90/90  16s 174ms/step - accuracy: 0.9219 - loss: 0.18
Epoch 20/25
90/90  3s 31ms/step - accuracy: 0.9688 - loss: 0.0629
Epoch 21/25
90/90  16s 175ms/step - accuracy: 0.9100 - loss: 0.19
Epoch 22/25
90/90  3s 32ms/step - accuracy: 0.9062 - loss: 0.1844
Epoch 23/25
90/90  16s 178ms/step - accuracy: 0.9156 - loss: 0.18
Epoch 24/25
90/90  3s 32ms/step - accuracy: 0.8125 - loss: 0.3259
Epoch 25/25
90/90  16s 180ms/step - accuracy: 0.9226 - loss: 0.17

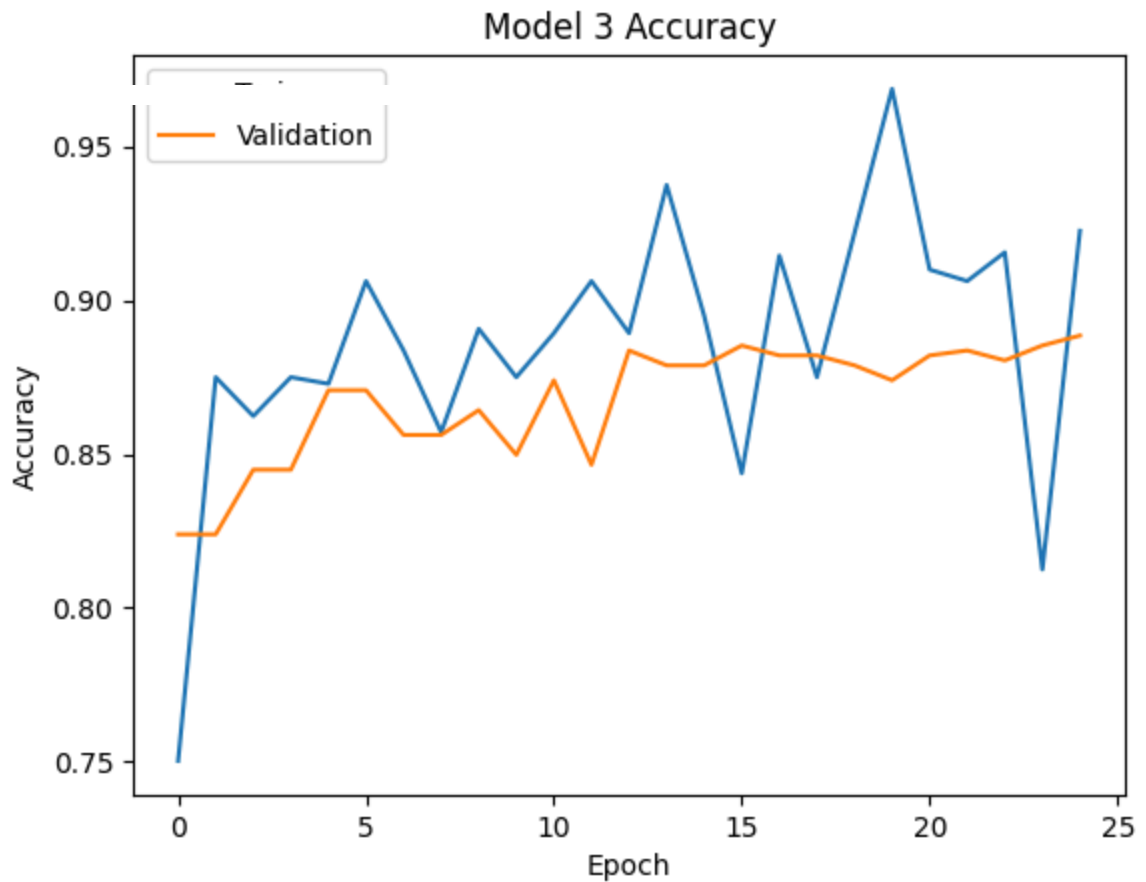
```

```

plt.plot(history_3.history['accuracy']) #Complete the code to plot the t
plt.plot(history_3.history['val_accuracy']) #Complete the code to plot t
plt.title('Model 3 Accuracy') #Complete the code to define the title for

```

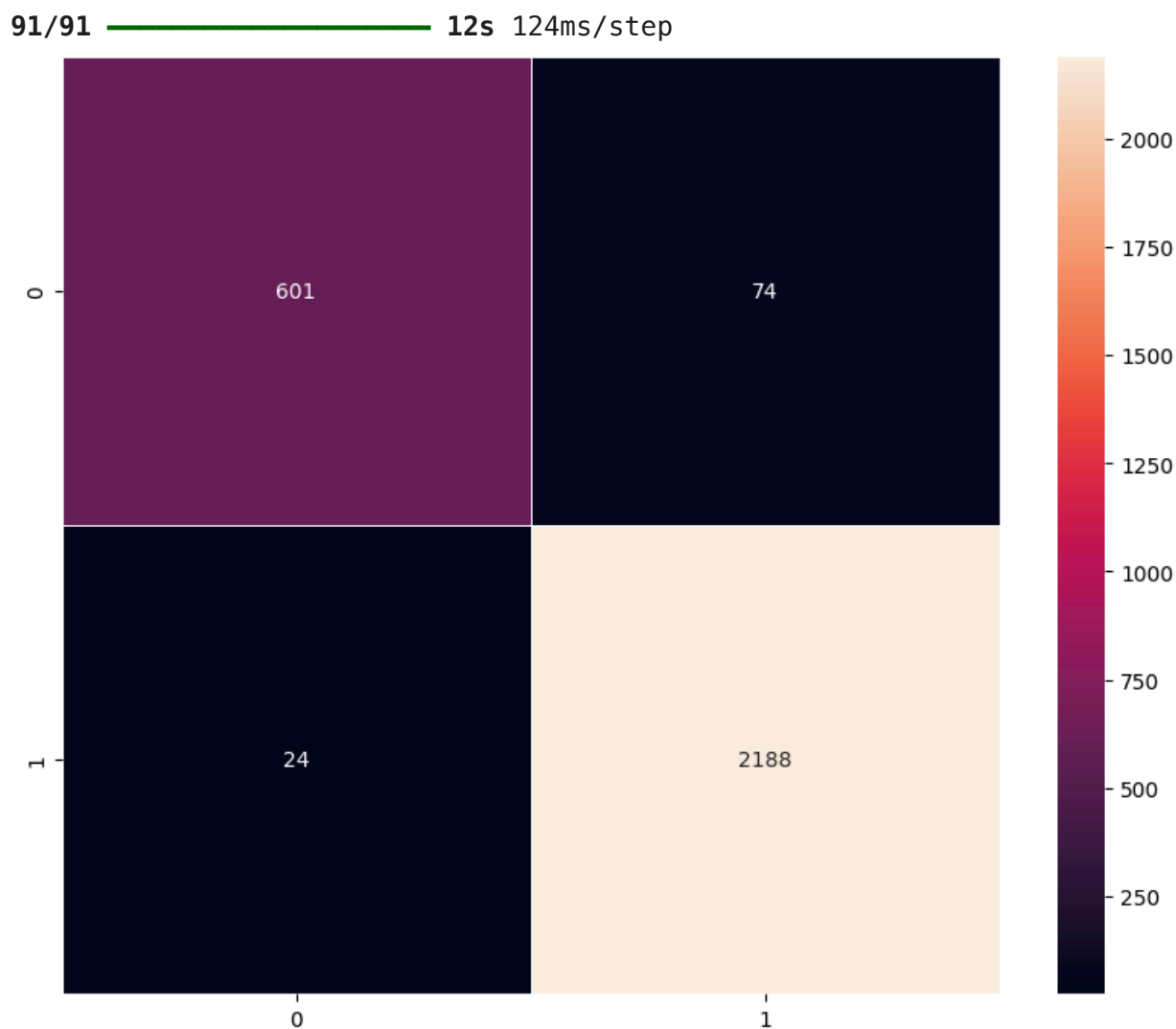
```
plt.ylabel('Accuracy') #Complete the code to define the label for the y-
plt.xlabel('Epoch') #Complete the code to define the label for the x-axi
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()
```



```
model_3_train_perf = model_performance_classification(model_3, X_train_n
print("Train performance metrics")
print(model_3_train_perf)
```

91/91 ————— **12s** 120ms/step
Train performance metrics
Accuracy Recall Precision F1 Score
0 0.966055 0.93976 0.964443 0.951356

```
plot_confusion_matrix(model_3,X_train_normalized,y_train)
```



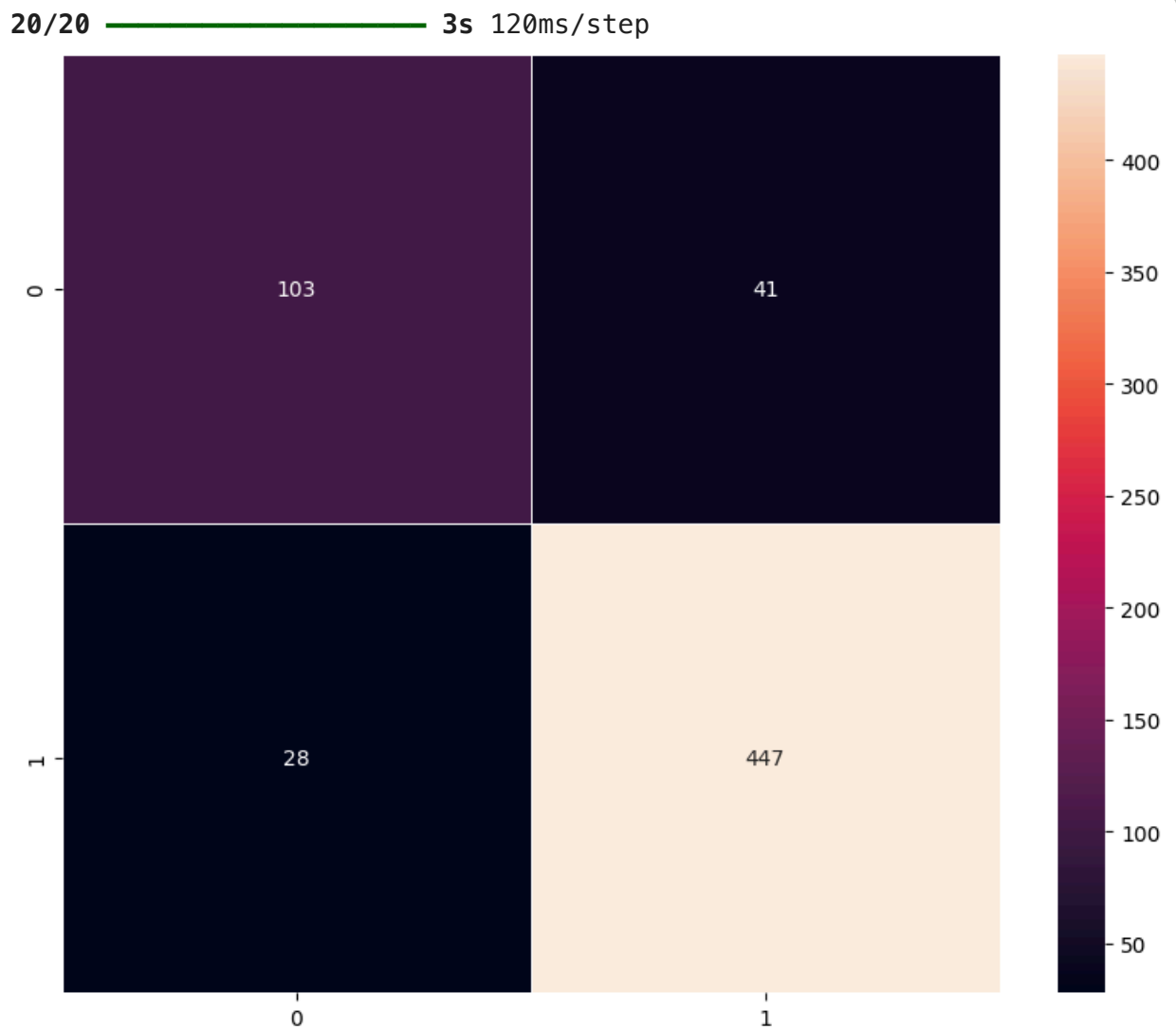
```
model_3_valid_perf = model_performance_classification(model_3, X_val_nor
print("Validation performance metrics")
print(model_3_valid_perf)
```

20/20 ————— 3s 121ms/step

Validation performance metrics

	Accuracy	Recall	Precision	F1 Score
0	0.88853	0.828165	0.851122	0.83872

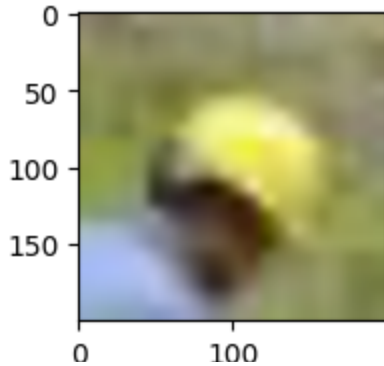
```
plot_confusion_matrix(model_3,X_val_normalized,y_val)
```



✓ Visualizing the predictions

```
plt.figure(figsize=(2,2))
plt.imshow(X_val[3]) #Complete the code to define the index
plt.show()
prediction = model_3.predict(X_val_normalized[3].reshape(1,200,200,3)) #
predicted_label = prediction[0][0]>0.5 # Extract the predicted class la
print('Predicted Label:', 1 if predicted_label else 0)
# Fix indexing issue in y_val
true_label = y_val.iloc[3] #Complete the code to define the index
print('True Label:', true_label)
```

```
plt.figure(figsize=(2,2))
plt.imshow(X_val[10]) #Complete the code to define the index
plt.show()
prediction = model_3.predict(X_val_normalized[10].reshape(1,200,200,3))
predicted_label = prediction[0][0]>0.5 # Extract the predicted class la
print('Predicted Label:', 1 if predicted_label else 0)
# Fix indexing issue in y_val
true_label = y_val.iloc[10] #Complete the code to define the index
print('True Label:', true_label)
```

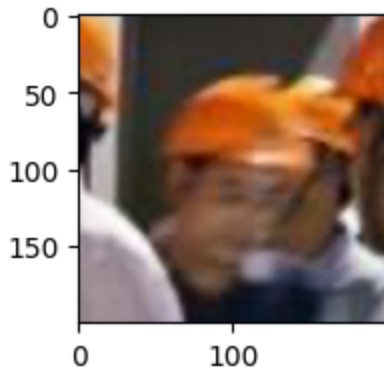


1/1 ————— 0s 87ms/step

Predicted Label: 1

True Label: label 1

Name: 4018, dtype: int64



1/1 ————— 0s 82ms/step

Predicted Label: 1

True Label: label 1

Name: 3988, dtype: int64

Model 4: Transfer Learning with VGG-16 (Base + FFNN + Data Augmentation)

- In most of the real-world case studies, it is challenging to acquire a large number of images and then train CNNs.
- To overcome this problem, one approach we might consider is **Data Augmentation**.

- CNNs have the property of **translational invariance**, which means they can recognise an object even if its appearance shifts translationally in some way. - Taking this attribute into account, we can augment the images using the techniques listed below
 - Horizontal Flip (should be set to True/False)
 - Vertical Flip (should be set to True/False)
 - Height Shift (should be between 0 and 1)
 - Width Shift (should be between 0 and 1)
 - Rotation (should be between 0 and 180)
 - Shear (should be between 0 and 1)
 - Zoom (should be between 0 and 1) etc.

Remember, **data augmentation should not be used in the validation/test data set.**

```
model_4 = Sequential()

# Adding the convolutional part of the VGG16 model from above
model_4.add(vgg_model)

# Flattening the output of the VGG16 model because it is from a convolut
model_4.add(Flatten())

#Adding the Feed Forward neural network
model_4.add(Dense(256,activation='relu')) #Complete the code to define t
model_4.add(Dropout(rate=0.5)) #Complete the code to define the dropout
model_4.add(Dense(128,activation='relu')) #Complete the code to define t

# Adding a dense output layer
model_4.add(Dense(1, activation='sigmoid')) #Complete the code to define
```

```
opt=Adam(learning_rate=0.001)
# Compile model
model_4.compile(optimizer=opt,    loss=keras.losses.BinaryCrossentropy(),
```

```
# Generating the summary of the model
model_4.summary()
```


Model: "sequential_6"

Layer (type)	Output Shape	Param
vgg16 (Functional)	(None, 6, 6, 512)	14,714,688
flatten_6 (Flatten)	(None, 18432)	
dense_12 (Dense)	(None, 256)	4,718,848
dropout_2 (Dropout)	(None, 256)	
dense_13 (Dense)	(None, 128)	32,896
dense_14 (Dense)	(None, 1)	12

Total params: 19,466,561 (74.26 MB)**Trainable params:** 4,751,873 (18.13 MB)**Non-trainable params:** 14,714,688 (56.13 MB)

```

# Applying data augmentation
# Applying data augmentation
train_datagen = ImageDataGenerator(
    rotation_range=20,          # Complete the code to define the range fo
    fill_mode='nearest',
    width_shift_range=0.2,      # Complete the code to define the width s
    height_shift_range=0.2,     # Complete the code to define the height
    shear_range=0.2,           # Complete the code to define the shear r
    zoom_range=0.2,            # Complete the code to define the zoom ra
)

```

```

history_4 = model_4.fit(train_datagen.flow(X_train_normalized,y_train,
                                             batch_size=32, #Complete the code
                                             seed=42,
                                             shuffle=False),
                        epochs=epochs,
                        steps_per_epoch=X_train_normalized.shape[0] // batch
                        validation_data=(X_val_normalized,y_val),
                        verbose=1)

```

```

Epoch 1/20
90/90 ━━━━━━━━━━━ 44s 471ms/step - accuracy: 0.7299 - loss: 0.61
Epoch 2/20
90/90 ━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.8125 - loss: 0.4402
Epoch 3/20
90/90 ━━━━━━━━━━━ 41s 460ms/step - accuracy: 0.7744 - loss: 0.46
Epoch 4/20
90/90 ━━━━━━━━━━━ 3s 27ms/step - accuracy: 0.8125 - loss: 0.3948
Epoch 5/20
90/90 ━━━━━━━━━━━ 41s 461ms/step - accuracy: 0.7926 - loss: 0.42
Epoch 6/20
90/90 ━━━━━━━━━━━ 3s 28ms/step - accuracy: 0.9062 - loss: 0.2363
Epoch 7/20
90/90 ━━━━━━━━━━━ 42s 462ms/step - accuracy: 0.8046 - loss: 0.41

```

```

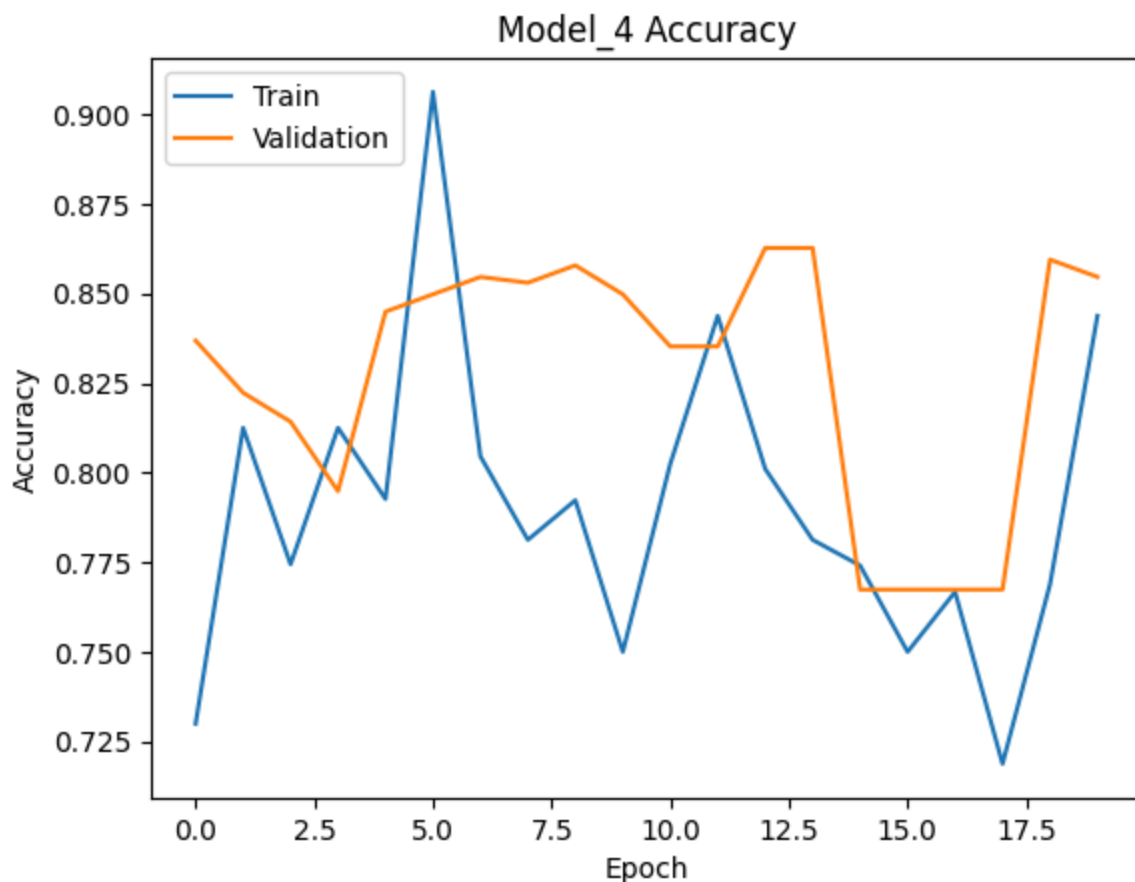
Epoch 8/20
90/90 ————— 3s 28ms/step - accuracy: 0.7812 - loss: 0.4635
Epoch 9/20
90/90 ————— 41s 458ms/step - accuracy: 0.7923 - loss: 0.39
Epoch 10/20
90/90 ————— 3s 28ms/step - accuracy: 0.7500 - loss: 0.4671
Epoch 11/20
90/90 ————— 40s 448ms/step - accuracy: 0.8025 - loss: 0.38
Epoch 12/20
90/90 ————— 6s 58ms/step - accuracy: 0.8438 - loss: 0.3069
Epoch 13/20
90/90 ————— 40s 440ms/step - accuracy: 0.8011 - loss: 0.39
Epoch 14/20
90/90 ————— 3s 29ms/step - accuracy: 0.7812 - loss: 0.4859
Epoch 15/20
90/90 ————— 42s 466ms/step - accuracy: 0.7741 - loss: 0.40
Epoch 16/20
90/90 ————— 3s 28ms/step - accuracy: 0.7500 - loss: 0.3992
Epoch 17/20
90/90 ————— 41s 452ms/step - accuracy: 0.7667 - loss: 0.41
Epoch 18/20
90/90 ————— 3s 29ms/step - accuracy: 0.7188 - loss: 0.4615
Epoch 19/20
90/90 ————— 41s 459ms/step - accuracy: 0.7688 - loss: 0.38
Epoch 20/20
90/90 ————— 3s 29ms/step - accuracy: 0.8438 - loss: 0.3059

```

```

plt.plot(history_4.history['accuracy']) #Complete the code to plot the t
plt.plot(history_4.history['val_accuracy']) #Complete the code to plot t
plt.title('Model_4 Accuracy') #Complete the code to define the title for
plt.ylabel('Accuracy') #Complete the code to define the label for the y-
plt.xlabel('Epoch') #Complete the code to define the label for the x-axi
plt.legend(['Train', 'Validation'], loc='upper left')
plt.show()

```



```
model_4_train_perf = model_performance_classification(model_4, X_train_n
print("Train performance metrics")
print(model_4_train_perf)
```

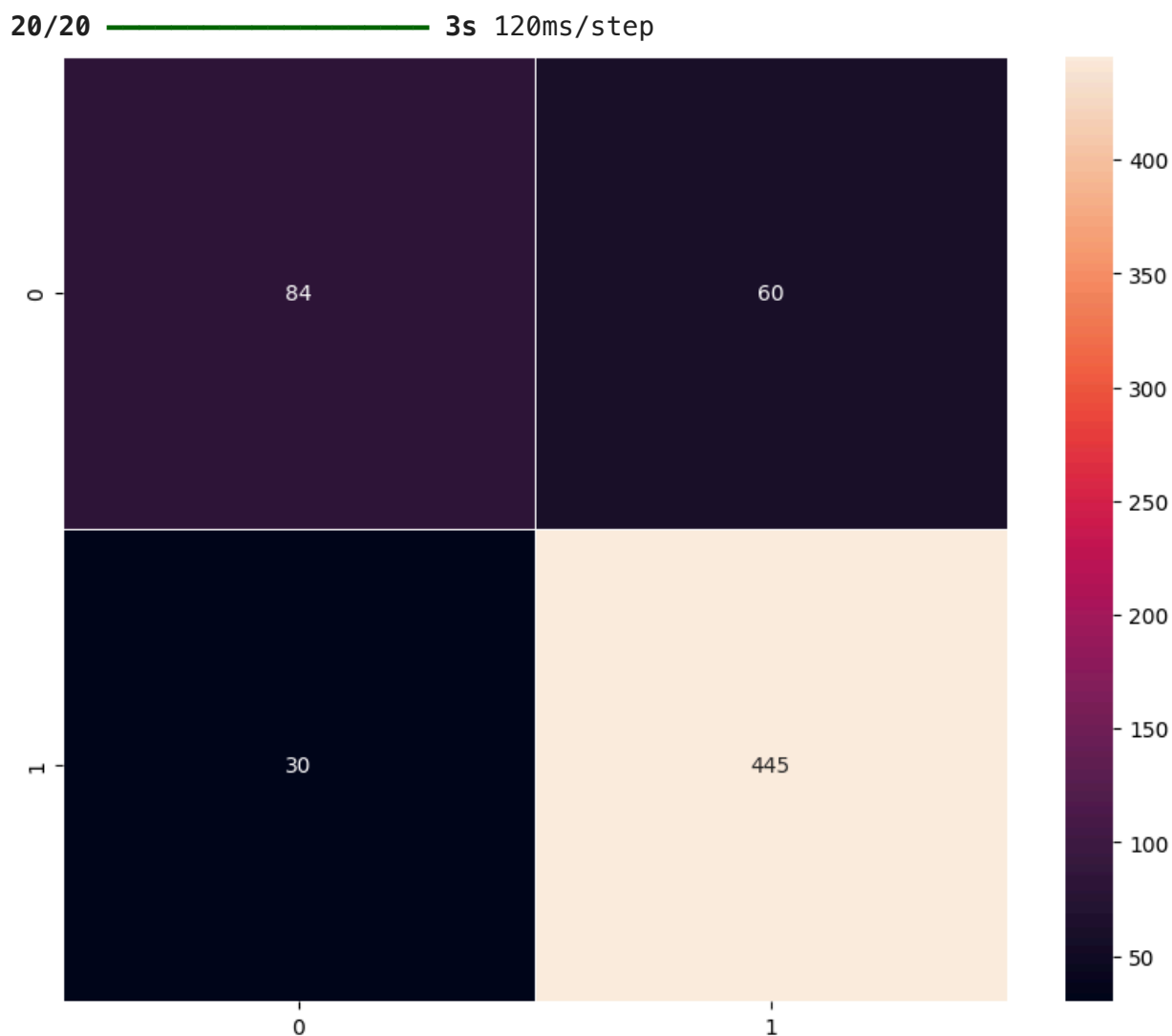
91/91 ————— **13s** 128ms/step
Train performance metrics
Accuracy Recall Precision F1 Score
0 0.897471 0.840961 0.86521 0.852102

```
plot_confusion_matrix(model_4,X_train_normalized,y_train)
```

```
model_4_valid_perf = model_performance_classification(model_4, X_val_nor
print("Validation performance metrics")
print(model_4_valid_perf)
```

20/20 ————— **3s** 123ms/step
Validation performance metrics
Accuracy Recall Precision F1 Score
0 0.854604 0.760088 0.809015 0.779663

```
plot_confusion_matrix(model_4,X_val_normalized,y_val)
```



✓ Visualizing the predictions

```
plt.figure(figsize=(2,2))
plt.imshow(X_val[11]) #Complete the code to define the index
plt.show()
prediction = model_4.predict(X_val_normalized[11].reshape(1,200,200,3))
predicted_label = prediction[0][0]>0.5 # Extract the predicted class 1
print('Predicted Label:', 1 if predicted_label else 0)
# Fix indexing issue in y_val
true_label = y_val.iloc[11] #Complete the code to define the index
print('True Label:', true_label)
```